

Подпрограммы

Понятие вспомогательного алгоритма

- В соответствии с методами структурного программирования построение программ производится методом постепенной детализации алгоритмов.
- Некоторые части алгоритма могут выделяться в виде вспомогательных алгоритмов, решающих вспомогательные подзадачи.
- Для реализации вспомогательных алгоритмов служат подпрограммы.
- **Подпрограмма** — самостоятельный фрагмент программы, оформленный в виде, допускающем многократное обращение к нему из разных точек программы.
- **Обращение к подпрограмме** — переход к выполнению подпрограммы с заданием информации, необходимой для ее выполнения и возврата.

Назначение подпрограмм

- избавляют от необходимости многократно повторять в тексте программы аналогичные фрагменты;
- улучшают структуру программы, облегчая ее понимание;
- повышают устойчивость к ошибкам программирования и непредвиденным последствиям при модификации программы.

Процедуры и функции

- **Вызовом подпрограммы** называется обращение к ней по имени с целью её использования. Вызов — это запуск подпрограммы.
- Существует два вида подпрограмм: процедуры и функции.
- Разница между ними состоит в том, что функция через свое имя возвращает одно значение определенного типа и может, использоваться в выражениях наряду со встроенными функциями.

Способы включения подпрограмм в код

- Подпрограммы группируются в отдельных файлах, имеющих специальную структуру – **модулях**. Чтобы программа могла использовать подпрограммы из модуля, он должен быть подключен к программе:

```
#include "имя файла"
```

```
#include <имя модуля>
```

Модульное программирование

- **Модульное программирование** — такая технология программирования, когда алгоритм всей решаемой задачи разбивается на отдельные, логически завершённые части.
- Если эти части сложны для кодирования, то они снова разбиваются на более простые части. Этот процесс продолжается до тех пор, пока не получатся простые для программирования алгоритмы, предназначенные для реализации несложных действий.
- Основные принципы модульного программирования были сформулированы ещё в 60-х годах 20-го века.

Подпрограмма в C++

```
тип <имя подпрограммы>  
(<список параметров>)  
{  
    <локальные переменные>  
    <операторы>  
}
```

- Формально в языках C/C++ нет процедур, а **имеются только функции.**
- Если необходима процедура, то достаточно создать функцию, возвращающую тип **void**.

Процедура

```
void <имя  процедуры> (<список  
    параметров>)  
{  
    <локальные  переменные>  
    <операторы>  
}
```

Функция

```
тип результата <имя функции>  
(<список параметров>)  
{  
  <локальные переменные>  
  <операторы>  
return <результат>;  
}
```

Функции на языке C++

1. Задать **прототип** функции;
2. Вызвать функцию в необходимом месте, например, в функции `main()`;
3. Дать определение функции.

- Любая функция должна быть **объявлена и определена.**
- Как и для других величин, объявлений может быть несколько, а определение только одно.
- **Объявление** функции (прототип, заголовок, сигнатура) задает ее имя, тип возвращаемого значения и список типов передаваемых параметров.

- Прототипы всех функций, используемых в программе, обычно записывают в начале текста программы **до определения функций.**
- Прототип необходим для того, чтобы у компилятора была полная информация о типе результата работы функции и о списке параметров, передаваемых в функцию.
- Когда в тексте программы встретится обращение к функции, то компилятор проверяет правильность её вызова, сверяясь с информацией из прототипа.

- **Вызов функции** возможен в любой функции, где это потребуется.
- Если тип результата функции — `void`, то вызов записывается отдельным оператором, в остальных случаях — в выражении того же типа, что и тип результата функции.
- **Определение функции** состоит из её заголовка и тела, записанного в виде блока.
- Допустимо записывать определение функций в любой последовательности.
- **Нельзя определять функцию внутри другой функции!!!**

Пример

```
#include <iostream>
using namespace std;

// 1)Прототип функции
double fmax(double a, double b);

int main()
{
    double a = 4, b = 3, max;
    cout << "a=" << a << " max=" << b << endl;

    // 2)Вызов функции
    max = fmax(a, b);

    cout << "max=" << max << endl;
    return 0;
}
```

```
// 3) Определение функции
double fmax(double x, double y)
{
    double max;
    if(x > y)
        max = x;
    else
        max = y;
    return max;
}
```

Формальные и фактические параметры

- В работе с подпрограммами приходится иметь дело с двумя видами параметров: **список фактических параметров и список формальных параметров.**
- Список параметров (фактических или формальных — всё равно) записывается в круглых скобках сразу за именем функции.
- Список параметров может быть пустым, **но скобки за именем функции обязательны!**

Пример: `void do_something();`

Формальные и фактические параметры

- Параметры, описанные в прототипе процедуры, называются **формальными**, а значения, которые присваиваются этим параметрам в процессе вызова – **фактическими** параметрами.

Формальные и фактические параметры

- Список фактических параметров — это те реальные данные, с помощью которых можно настроить алгоритм подпрограммы на обработку конкретных данных.
- Список формальных параметров — это по сути набор требований, которые предъявляет подпрограмма к передаваемым в неё данным.

Формальные и фактические параметры

- Между списками фактических и формальных параметров должно выдерживаться полное соответствие по количеству параметров, их типу, порядку следования и способу передачи в функцию.

Пример

//объявление целочисленной функции

```
int max(int, int);
```

```
int main()
```

```
{
```

```
int n = 5, m = 7;
```

```
int c = max(n, m); //вызов функции
```

```
}
```

```
int max(int a, int b) //заголовок ф-ции
```

//внутри нее переданные формальные

// параметры будут иметь имена **a** и **b**

Механизм вызова функции

- Перед вызовом функции сначала вычисляются значения выражений, используемых в качестве фактических параметров.
- Затем в программном стеке выделяется память для локальных переменных этой функции.

- **Программный стек** – это специальная область памяти, организованная по правилу “last in first out” (последний вошел, первым вышел) и создаваемая автоматически для каждого потока программы.
- Программный стек служит для хранения локальных переменных функции, передачи возвращаемого результата из функций и сохранения адреса возврата из функции.

- После расчета параметров и выделения места на стеке происходит запись параметров функции в стек.
- Потом в стеке запоминается адрес точки возврата – адрес в памяти оператора, к которому должно перейти управление после завершения функции.



- В итоге, управление передается в функцию, после чего выполняются операторы функции. При достижении оператора **return** вычисляется выражение – результат функции – и передается в точку возврата.
- В конце концов, управление передается в точку возврата, а память из-под локальных переменных функции освобождается.

Локальные и глобальные переменные

- Внутри функции можно объявлять свои переменные. Такие переменные, а также параметры функции являются **локальными**.
- Областью их действия является функция.
- При выходе из функции соответствующий участок стека освобождается, поэтому значения локальных переменных между вызовами одной и той же функции не сохраняются.
- Если этого требуется избежать, при объявлении локальных переменных используется модификатор **static**.

- При совместной работе функции должны обмениваться информацией.
- Это можно осуществить с помощью **глобальных переменных**, через параметры и через возвращаемое функцией значение.
- Глобальные переменные определяются вне любой функции в любом месте программы.

- Глобальные переменные видны во всех функциях, **где не описаны локальные переменные с теми же именами**, поэтому использовать их для передачи данных между функциями очень легко.
- Использовать этот способ не рекомендуется, поскольку это затрудняет отладку программы и препятствует помещению функций в библиотеки общего пользования.
- Нужно стремиться к тому, чтобы функции были максимально независимы, а их интерфейс полностью определялся прототипом функции, поэтому использовать глобальные переменные не рекомендуется.

```
#include <stdio.h>
```

```
int m = 0;
```

Глобальная область видимости
доступны все функции из `stdio.h` и
переменная `m`

```
void max_num(int a, int b){  
    m = b;  
    if (a > b)  
        m = a;  
}
```

Локальная область видимости
переменные `a` и `b` доступны и
существуют только внутри функции
`max_num`

```
int main(void) {  
    int x = 0, y = 0;  
  
    scanf("%d %d", &x, &y);  
  
    max_num(x,y);  
  
    printf("max(%d,%d) = %d\n",x,y,m);  
  
    return 0;  
}
```

Локальная область видимости
переменные `x` и `y` доступны до конца
функции `main`

Способы передачи данных в функции

В языке C++ данные в подпрограмму можно передавать тремя способами:

- по значению,
- по адресу,
- по ссылке.

Передача данных по значению

- Этот способ передачи данных в подпрограмму является основным и **действует по умолчанию.**
- Фактический параметр вычисляется в вызывающей функции и его значение передаётся на место формального параметра в вызываемой функции.
- На этом связь между фактическим и формальным параметрами прекращается.

- При передаче *по значению* в стек заносятся копии значений фактических параметров, и операторы функции работают с этими копиями.
- Доступа к исходным параметрам у функции нет, а, следовательно, нет возможности их изменить.

```
double fsum(double x, double eps);
```

```
int main()
```

```
{
```

```
    double x, s, eps = 1.0e-5;
```

```
    cout << "x = ";
```

```
    cin >> x;
```

```
    s = fsum(x, eps);
```

```
    cout << "s = " << s << endl;
```

```
    return 0;
```

```
}
```

```
double fsum(double x, double eps)
```

```
{
```

```
    double s = x, p = x, i, t = x * x;
```

```
    for(i = 3; fabs(p) > eps; i += 2)
```

```
    {
```

```
        p = -p * t / (i * (i - 1));
```

```
        s += p;
```

```
    }
```

```
    return s;
```

```
}
```

Пример 2

- Даны два числа, хранящиеся в переменных `a` и `b`. Используя подпрограмму, выполнить обмен содержимого ячеек этих переменных.

- **Через параметры, передаваемые по значению, нельзя вернуть результаты работы функции.**

Передача данных по адресу

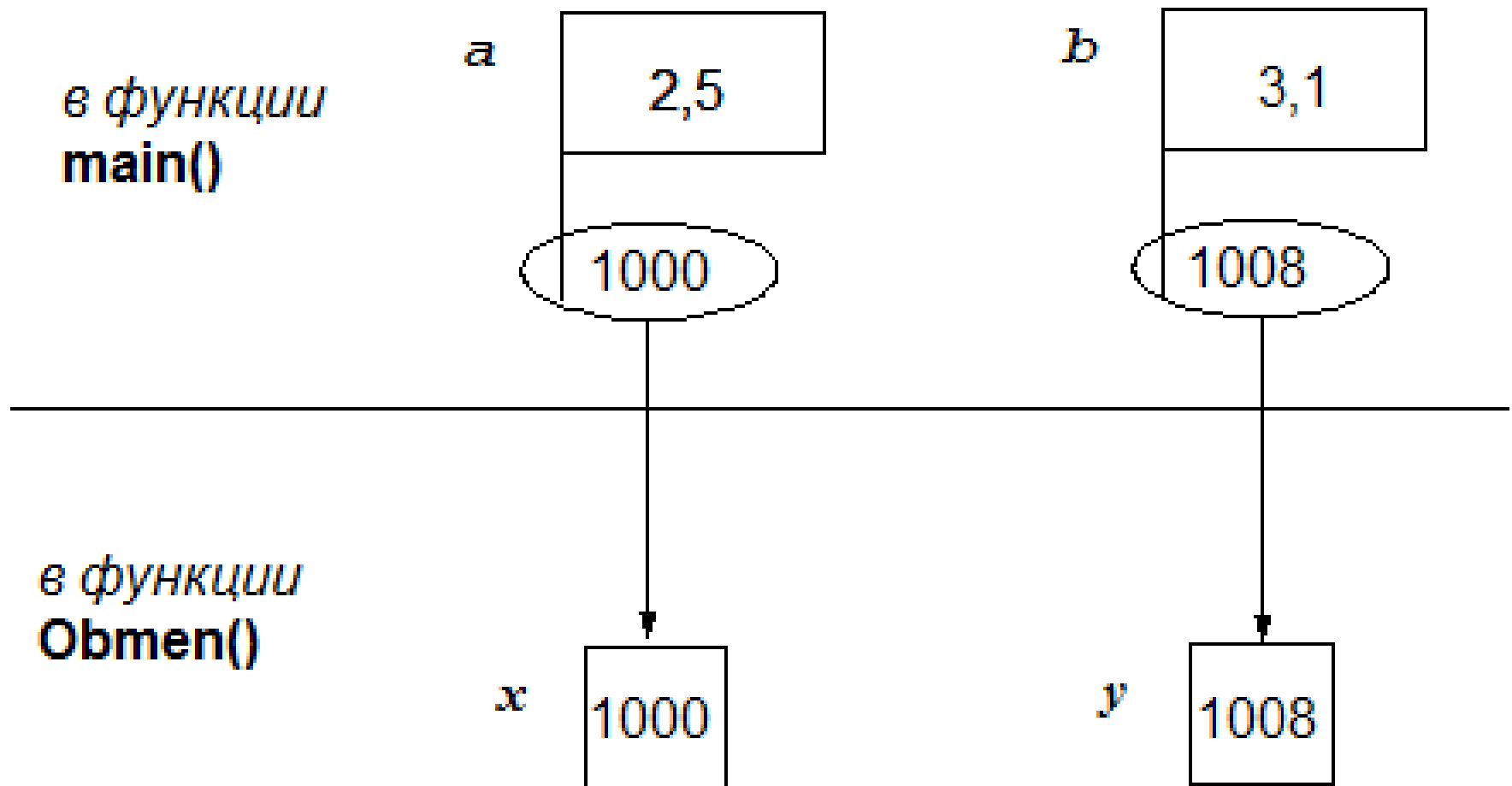
- По адресу в функцию всегда передаются массивы. Для массива это вообще единственный способ передачи данных в языках C/C++.
- Так же по адресу можно передать те простые объекты, которые являются выходными данными (или входными и выходными одновременно).
- В случае передачи данных по адресу фактический параметр может быть только переменной (константа или выражение не имеют адреса!).

- При передаче *по адресу* в стек заносятся копии адресов параметров, а функция осуществляет доступ к ячейкам памяти по этим адресам и может изменить исходные значения параметров.

Передача данных по адресу

- Для передачи параметров по адресу перед названием параметра ставится *.
- **void** Obmen (**double** *x, **double** *y) ;

Передача данных по адресу



Передача данных по ссылке

- Это ещё один из способов вернуть результат работы функции через список параметров.
- При передаче данных по ссылке в функцию, куда передаются данные, создаются синонимы исходных объектов. Поэтому работа в подпрограмме ведётся именно с исходными объектами.
- Если в подпрограмме ссылочная переменная изменит значение, то это сразу отразится на исходной переменной.
- В вызывающей функции параметр, передаваемый по ссылке, может быть только простой переменной любого известного типа.

Передача данных по ссылке

- Для объекта, передаваемого по ссылке, в списке формальных параметров указывается значок & перед именем переменной (работаем с ссылкой на объект), а далее в теле подпрограммы используется просто имя этой переменной.
- При вызове в списке фактических параметров задаётся имя нужной переменной.

Что выбирать?

- Для простых объектов передача данных по ссылке предпочтительнее, чем передача по адресу, так как в этом случае текст функции проще, легче читается, не требуется выполнять операции разыменования.
- К недостатку передачи данных по ссылке можно отнести то, что по вызову функции нельзя понять, что параметр передаётся именно по ссылке, а не по значению, и как следствие, этот параметр скорее всего изменяется функцией.
- Всегда нужно смотреть на запись прототипов функций, использованных в программе. Прототипы всегда доступны, даже если сами функции имеются только в виде объектных модулей.

Функции с параметрами по умолчанию

- Язык C++ позволяет присваивать параметрам функции значения по умолчанию в том случае, когда при вызове функции не указываются параметры.
- Значения по умолчанию задаются синтаксически сходным образом с инициализацией переменных.

Пример

```
void drawRectangle(  
int countSymb1 = 10,  
int countSymb2 = 10, char symb = '@')
```

```
void drawRectangle(  
int countSymb1, int countSymb2,  
char symb = '@')
```

- Определяться параметры по умолчанию должны **справа налево**.
- Например если из трех параметров необходимо задать только один по умолчанию, то он должен быть определен крайним справа.

Перегрузка функций

- С++ позволяет определять несколько функций с одинаковым именем в одной области.
- Такие функции называются **перегруженными**.
- Перегрузка функций в С++ используется, когда нужно сделать одно и то же действие с разными типами данных.

Пример

```
/* Функция max для целых чисел */  
int max(int num1, int num2)  
{  
    if (num1 > num2)  
        return num1;  
    return num2;  
}
```

```
/* Функция max для чисел с плавающей запятой */  
double max(double num1, double num2)  
{  
    if (num1 > num2)  
        return num1;  
    return num2;  
}
```

```
// Здесь будет использоваться первый вариант функции max  
int imax = max(1, 10);  
// А здесь - второй  
double dmax = max(1.0, 20.0);
```

Функции с переменным числом параметров

- В C++ можно создавать функции с переменным числом параметров.
- Параметры, их количество и типы, становятся известны только во время вызова функции.
- Формат описания функций с переменным числом параметров:

```
тип имя (<список явных параметров>, ...)  
{  
<тело функции>  
}
```

- После списка явных параметров следует необязательная запятая и **троеточие**, которое сообщает компилятору, что контроль типов и количества параметров при вызове функции не следует.
- При создании функций с переменным числом параметров необходимо предусмотреть способ определения количества параметров и их типов.
- Используется для этих целей два способа:
 1. Один из параметров определяет число параметров функции.
 2. В списке явных параметров задается параметр, указывающий на конец списка параметров.

- Если не один из этих способов не используется, тогда можно использовать ***специальный набор макроопределений***.
- Параметры функции помещаются в стек, при этом первый параметр оказывается в вершине стека.
- Переход от одного параметра к другому осуществляется с помощью указателей.
- Для использования в программе функций с переменным числом параметров, при вызове которых можно использовать параметры разных типов, удобно использовать специальный набор макроопределений (заголовочный файл **`stdarg.h`**).

Пример

```
#include <stdio.h>
#include <conio.h>
#include <iostream>
void main()
{
    int sum(int n,...); //прототип функции с переменным
                        //числом параметров
    cout << "\n 4+6=" << sum(2,4, 6);
    cout << "\n 1+2+3+4+5+6=" << sum(6,1,2,3,4,5,6);
    cout << "\n Parametrov net.Summa ravna:" << sum(0);
    getch();
}
int sum(int n,...)//n-число суммируемых параметров
{
    int s=0;
    int *p = &n; //получение адреса параметров в стеке
    for(int i=1; i<=n; i++)
        s+=*++p; //суммируем числа
    return s;
}
```

Шаблоны функций

- Шаблоны (англ. template) — средство языка C++, предназначенное для кодирования обобщённых алгоритмов, без привязки к некоторым параметрам (например, типам данных или значениям по умолчанию).

Пример

Для каждого типа аргументов
своя перегрузка???

```
1  #include <iostream>
2
3  int add(int x, int y)
4  {
5      return x + y;
6  }
7  double add(double x, double y)
8  {
9      return x + y;
10 }
11 int main()
12 {
13     double n1 = add(4.7, 5.3); // 10
14     int n2 = add(4, 5);        // 9
15
16     std::cout << "n1: " << n1 << std::endl;
17     std::cout << "n2: " << n2 << std::endl;
18
19     return 0;
20 }
```

- Шаблоны функций представляют некоторый образец, по которому можно создать конкретную функцию, специфическую для определенного типа:

template<typename T>

тип имя (T аргумент1, T аргумент2)

{

<Тело функции>

return res;

}

Исправленный пример

```
3  template<typename T>
4  T add(T x, T y)
5  {
6      return x + y;
7  }
8  int main()
9  {
10     double a1 = 4.7, b1 = 5.3;
11     double n1 = add(a1, b1);
12
13     int a2 = 4, b2 = 5;
14     int n2 = add(a2, b2);
15
16     short a3 = 3, b3 = 2;
17     short n3 = add(a3, b3);
18
19     std::cout << "n1: " << n1 << std::endl;
20     std::cout << "n2: " << n2 << std::endl;
21     std::cout << "n3: " << n3 << std::endl;
```